Guatemala, 9 de marzo de 2026

CURSO: PROGRAMACIÓN PARA DATA SCIENCE
CATEDRÁTICO: MSC. SERGIO MOLINA CASTELLANOS
DOCUMENTO: LABORATORIO NO. 9

NOMBRE: DIEGO JAVIER MORALES MONZÓN **CARNÉ:** 1132119
NOMBRE: ESTUARDO JOSÉ VILLEDA NAVARRO **CARNÉ:** 1003519
NOMBRE: JORGE WALDEMAR GARCÍA BALDIZÓN **CARNÉ:** 1220019
NOMBRE: JOSÉ DANIEL DE LEÓN CHANG **CARNÉ:** 1170419

INSTRUCCIONES: Suba la tarea de la semana.

Respuestas

Parte 1: Cargar y Explorar un Dataset

- Cargar el famoso conjunto de datos Iris (que está disponible en Scikit-learn), y hacer un análisis preliminar de sus características.
  - Utiliza load_iris de sklearn.datasets para cargar el conjunto de datos.
  - Convierte los datos en un DataFrame de Pandas para facilitar su manipulación.
  - Muestra las primeras 5 filas del dataset y describe las variables.

```
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   target             150 non-null    int64
```

Parte 2: División del Conjunto de Datos

Dividir el conjunto de datos en conjuntos de entrenamiento y prueba.

- Usa train_test_split para dividir los datos en 80% de entrenamiento y 20% de prueba.
- Separa las características (features) y el objetivo (target)

```
# Separar variables
X = df.drop("target", axis=1)
y = df["target"]

# División 80% entrenamiento - 20% prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Parte 3: Escalado de Características

- Normalizar las características utilizando un escalador estándar para que todas las características tengan una media de 0 y desviación estandar de 1.
    - Usa StandardScaler de scikit-learn para escalar las características
    - Aplica el escalador a los conjuntos de entrenamiento y prueba

```
scaler = StandardScaler()

# Ajustar SOLO con entrenamiento
X_train_scaled = scaler.fit_transform(X_train)

# Transformar test
X_test_scaled = scaler.transform(X_test)
```

Parte 4: Entrenar un Modelo de Regresión Logística

Entrenar un modelo de Regresion Logística para clasificar las flores del conjunto de datos Iris.

- Usa LogisiticRegression para entrenar el modelo con los datos de entrenamiento escalados.
- Haz predicciones con los datos de prueba.

```python
# PARTE 4: REGRESIÓN LOGÍSTICA
# ====================================================

model_lr = LogisticRegression()

# Entrenar modelo
model_lr.fit(X_train_scaled, y_train)

# Predicciones
y_pred_lr = model_lr.predict(X_test_scaled)

# ====================================================
# PARTE 5: EVALUACIÓN
# ====================================================
print("\n===== PARTE 5: EVALUACIÓN =====")

print("\nAccuracy:")
print(accuracy_score(y_test, y_pred_lr))

print("\nMatriz de confusión:")
print(confusion_matrix(y_test, y_pred_lr))

print("\nReporte de clasificación:")
print(classification_report(y_test, y_pred_lr))
```

Parte 5: Evaluación del Modelo

- Evaluar el rendimiento del modelo utilizando varias métricas de clasificación
  - Calcula la precisión del modelo utilizando accuracy_score
  - Genera una matriz de confusión para evaluar el rendimiento por clase.
  - Genera un informe de clasificación con las métricas de precisión, recall y f1-score.

```
Accuracy:
1.0

Matriz de confusión:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

```
Reporte de clasificación:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

Parte 6: Experimentar con otros Modelos de clasificacion
Entrenar otros algoritmos de clasificacion para comparar los resultados con el modelo de regresión logística

- Entrena un modelo de K-Nearest Neighbors (KNN) y un modelo de Support Vector Machine (SVM)
- Evalúa sus rendimientos y compáralos con la regresión logística.

```
KNN Accuracy:
1.0

SVM Accuracy:
1.0
```

```python
# ----- KNN -----
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
y_pred_knn = knn.predict(X_test_scaled)

print("\nKNN Accuracy:")
print(accuracy_score(y_test, y_pred_knn))


# ----- SVM -----
svm = SVC()
svm.fit(X_train_scaled, y_train)
y_pred_svm = svm.predict(X_test_scaled)

print("\nSVM Accuracy:")
print(accuracy_score(y_test, y_pred_svm))
```

Parte 7: Guardar y Cargar un Modelo Entrenado

- Guardar el modelo entrenado y cargarlo nuevamente para hacer predicciones.
    - Usa joblib para guardar el modelo en un archivo.
    - Carga el modelo guardado y verifica si puede hacer predicciones correctamente.

```
--- RESULTADO DE PREDICCIÓN ---
Predicción (número): 1
Predicción (nombre): versicolor

Valor real (número): 1
Valor real (nombre): versicolor

El modelo acertó

===== RESULTADOS DEL MODELO =====
   Real  Predicción Real Nombre Predicción Nombre  Correcto
0     1           1  versicolor        versicolor      True
1     0           0      setosa            setosa      True
2     2           2   virginica         virginica      True
3     1           1  versicolor        versicolor      True
4     1           1  versicolor        versicolor      True
5     0           0      setosa            setosa      True
1     0           0      setosa            setosa      True
2     2           2   virginica         virginica      True
3     1           1  versicolor        versicolor      True
4     1           1  versicolor        versicolor      True
1     0           0      setosa            setosa      True
2     2           2   virginica         virginica      True
3     1           1  versicolor        versicolor      True
1     0           0      setosa            setosa      True
2     2           2   virginica         virginica      True
1     0           0      setosa            setosa      True
2     2           2   virginica         virginica      True
1     0           0      setosa            setosa      True
2     2           2   virginica         virginica      True
1     0           0      setosa            setosa      True
2     2           2   virginica         virginica      True
1     0           0      setosa            setosa      True
1     0           0      setosa            setosa      True
1     0           0      setosa            setosa      True
1     0           0      setosa            setosa      True
1     0           0      setosa            setosa      True
1     0           0      setosa            setosa      True
1     0           0      setosa            setosa      True
1     0           0      setosa            setosa      True
2     2           2   virginica         virginica      True
3     1           1  versicolor        versicolor      True
4     1           1  versicolor        versicolor      True
1     0           0      setosa            setosa      True
2     2           2   virginica         virginica      True
3     1           1  versicolor        versicolor      True
1     0           0      setosa            setosa      True
2     2           2   virginica         virginica      True
1     0           0      setosa            setosa      True
1     0           0      setosa            setosa      True
1     0           0      setosa            setosa      True
1     0           0      setosa            setosa      True
1     0           0      setosa            setosa      True
1     0           0      setosa            setosa      True
2     2           2   virginica         virginica      True
3     1           1  versicolor        versicolor      True
4     1           1  versicolor        versicolor      True
5     0           0      setosa            setosa      True
6     1           1  versicolor        versicolor      True
7     2           2   virginica         virginica      True
8     1           1  versicolor        versicolor      True
9     1           1  versicolor        versicolor      True
```

Código Completo

```
# ============================================================
# LABORATORIO 9 - SCIKIT LEARN
# ============================================================

try:
    # ======================================================
    # PASO 1: IMPORTAR LIBRERÍAS
    # ======================================================
    import pandas as pd
    import numpy as np

    from sklearn.datasets import load_iris
    from sklearn.model_selection import train_test_split
    from sklearn.preprocessing import StandardScaler

    from sklearn.linear_model import LogisticRegression
    from sklearn.neighbors import KNeighborsClassifier
    from sklearn.svm import SVC

    from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

    import joblib

    # ======================================================
    # PARTE 1: CARGAR Y EXPLORAR DATASET
    # ======================================================
    print("===== PARTE 1: CARGA Y EXPLORACIÓN =====")

    # Cargar dataset Iris
    iris = load_iris()

    # Convertir a DataFrame
    df = pd.DataFrame(iris.data, columns=iris.feature_names)

    # Agregar columna objetivo
    df["target"] = iris.target

    # Mostrar primeras filas
    print("\nPrimeras 5 filas:")
```

```python
print(df.head())

# Información general
print("\nInformación del dataset:")
print(df.info())

# Estadísticas
print("\nEstadísticas descriptivas:")
print(df.describe())


# ======================================================
# PARTE 2: DIVISIÓN DE DATOS
# ======================================================

# Separar variables
X = df.drop("target", axis=1)
y = df["target"]

# División 80% entrenamiento - 20% prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


# ======================================================
# PARTE 3: ESCALADO DE CARACTERÍSTICAS
# ======================================================
scaler = StandardScaler()

# Ajustar SOLO con entrenamiento
X_train_scaled = scaler.fit_transform(X_train)

# Transformar test
X_test_scaled = scaler.transform(X_test)

# ======================================================
# PARTE 4: REGRESIÓN LOGÍSTICA
# ======================================================

model_lr = LogisticRegression()
```

```python
# Entrenar modelo
model_lr.fit(X_train_scaled, y_train)

# Predicciones
y_pred_lr = model_lr.predict(X_test_scaled)

# ======================================================
# PARTE 5: EVALUACIÓN
# ======================================================
print("\n===== PARTE 5: EVALUACIÓN =====")

print("\nAccuracy:")
print(accuracy_score(y_test, y_pred_lr))

print("\nMatriz de confusión:")
print(confusion_matrix(y_test, y_pred_lr))

print("\nReporte de clasificación:")
print(classification_report(y_test, y_pred_lr))


# ======================================================
# PARTE 6: OTROS MODELOS (KNN y SVM)
# ======================================================
print("\n===== PARTE 6: OTROS MODELOS =====")

# ----- KNN -----
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
y_pred_knn = knn.predict(X_test_scaled)

print("\nKNN Accuracy:")
print(accuracy_score(y_test, y_pred_knn))


# ----- SVM -----
svm = SVC()
svm.fit(X_train_scaled, y_train)
y_pred_svm = svm.predict(X_test_scaled)

print("\nSVM Accuracy:")
print(accuracy_score(y_test, y_pred_svm))
```

```python
    # ======================================================
    # PARTE 7: GUARDAR Y CARGAR MODELO
    # ======================================================
    print("\n===== GUARDAR Y CARGAR =====")

    # Guardar modelo
    joblib.dump(model_lr, "modelo_iris.pkl")

    # Cargar modelo
    modelo_cargado = joblib.load("modelo_iris.pkl")

    # Probar predicción (primer dato del test)
    ejemplo = [X_test_scaled[0]]
    pred = modelo_cargado.predict(ejemplo)

    # ================================
    # MOSTRAR RESULTADOS
    # ================================

    print("\n--- RESULTADO DE PREDICCIÓN ---")

    # Predicción del modelo
    print("Predicción (número):", pred[0])
    print("Predicción (nombre):", iris.target_names[pred[0]])

    # Valor real
    valor_real = y_test.iloc[0]
    print("\nValor real (número):", valor_real)
    print("Valor real (nombre):", iris.target_names[valor_real])

    # Comparación
    if pred[0] == valor_real:
        print("\nEl modelo acertó")
    else:
        print("\nEl modelo falló")

# ==========================================
# VER RESULTADOS EN TABLA
# ==========================================
```

```
    resultados = pd.DataFrame({
        "Real": y_test.values,
        "Predicción": y_pred_lr
    })

    # Convertir a nombres
    resultados["Real Nombre"] = resultados["Real"].apply(lambda x:
iris.target_names[x])
    resultados["Predicción Nombre"] =
resultados["Predicción"].apply(lambda x: iris.target_names[x])

    # Comparación
    resultados["Correcto"] = resultados["Real"] ==
resultados["Predicción"]

    print("\n===== RESULTADOS DEL MODELO =====")
    print(resultados.head(10))
# ============================================================
# MANEJO DE ERRORES
# ============================================================
except ImportError as e:
    print("Error: Falta instalar una librería")
    print(e)

except FileNotFoundError as e:
    print("Error: Archivo no encontrado")
    print(e)

except Exception as e:
    print("Error inesperado:")
    print(e)
```